Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	11
Members	153823257, 50155044, 152286617
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/katariinatuni/rec-sys-project

How to use this template

- *This is one cumulative report. Extend and revise the same document at each project stage.*
- *Maximum length: 4 pages at Part 1, 8 pages at Part 2, and 12 pages for the final report, excluding references.*
- *Keep the section headings below. You may add subsections where useful.*
- *All explanations and justifications requested in the project specification belong in this report.*
- *The Results Sheet contains the complete standardized numerical record. In the report, include only the tables and figures needed to support your analysis.*
- *Delete all italic instructional text before the final submission.*

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

All referenced numbers can be seen from the output data_stats.json and all figures are available as per readme.md. The 1,000,209 ratings fill 4.47 % of the 6,040 x 3,706 matrix spanning 25 April 2000 to 28 Feb 2003. The ratings are very strongly concentrated towards the beginning and with highly concentrated times of new ratings as seen in Figure 1

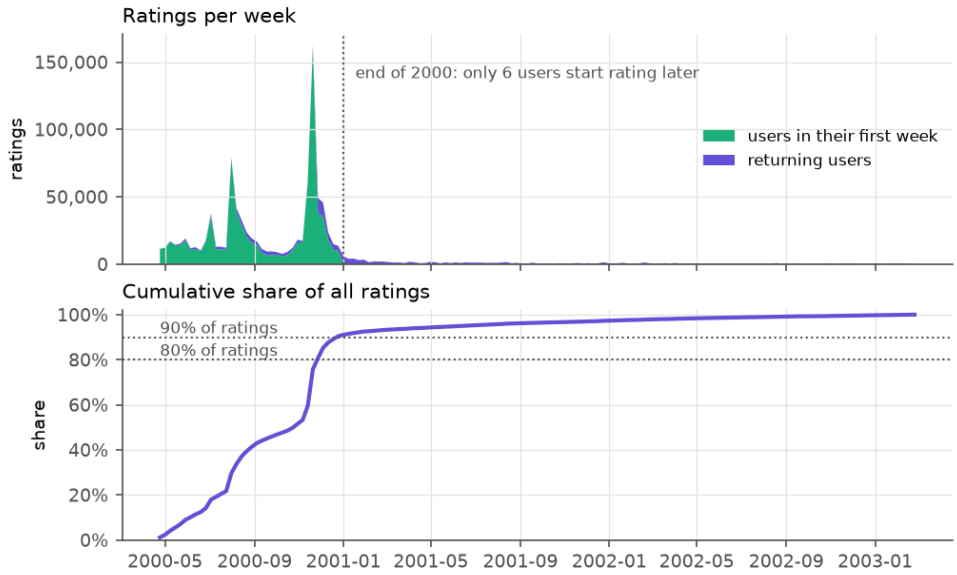


Figure 1 - Activity over time

The 3706 items rated are distributed among 6040 users rather unevenly. The mean user has done 166 ratings while the median is 96 ratings. The most ratings one user has done is 2314 and the most one movie has received was 3428 while the median is 123.5 and mean is 270 ratings per movie. This can also be seen from the gini for movies being 0.63. The most rated 20 % of movies receive 65 % of all ratings while the bottom 50 % receive only 7.6 %. Interestingly the users tend to concentrate rating good movies because the ratings skew positive with 57.5 % being 4 or 5.[1] The concentration of ratings can be seen from Figure 2 where it is clearly seen that the ratings are especially concentrated on the most reviewed films and some active users. In the three spike weeks of 2000 the share of ratings done by new users[2] was more than 95 %.

This phenomenon sprang another question about how far the ratings concentrate to the beginning of each users' activity. The share of ratings done in the first week of activity of all users was 81.8 %. The first day had 72.4 % and the first hour 52.0 % of all ratings. Per user this means that each user rated on average 86, and 56 on median, movies within the first hour. There is also a hard floor of 20 ratings per user.

[1] See rating_distribution.png

[2] Ratings calculated 7 days from the beginning of the first rating of each users. See explore.py user_start_stats

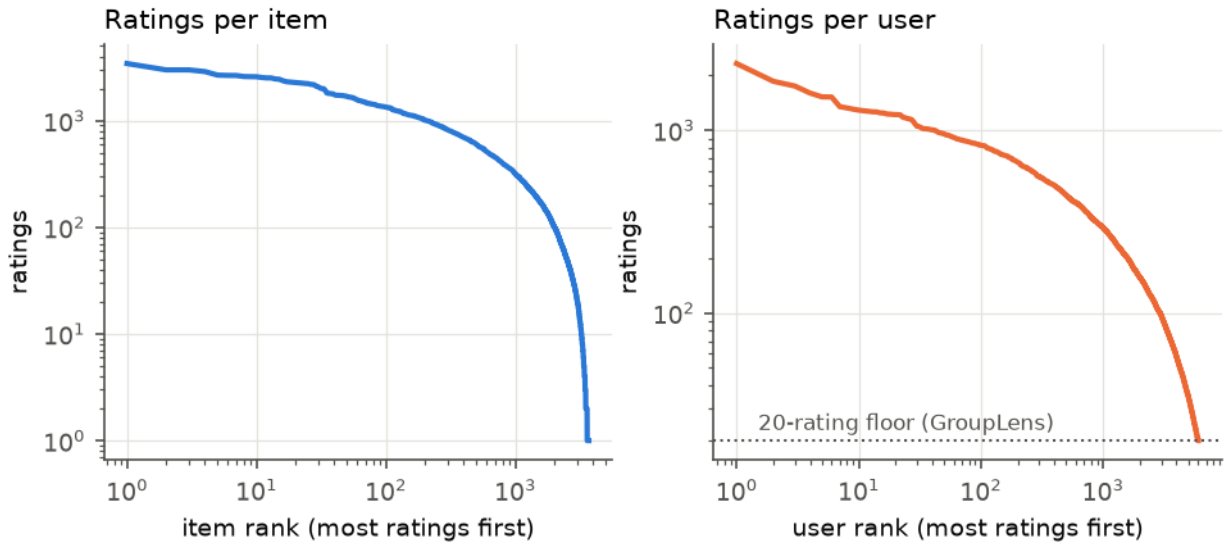


Figure 2 - Concentration of ratings

Diving into this further all ratings were grouped by user and timestamp. 77.4 % of ratings were added in batches of 2 or more. The number of reviews in a batch diminished by half on each step until 10 to 11 where it dropped sixfold. This was further tested by analysing timings between batches[3] and the sizes of the batches. Median times increased logically up to 10 reviews in a batch and then jumped unexpectedly.[4] 10 rating batches had also curiously a clear drop of mean stars to 3.13 stars.[5]

1.2 Preprocessing and feedback treatment

Table 1 - Thresholds

Threshold	Rule	Validation	Test
Relevance	held-out rating ≥ 4	55.1 % of ratings relevant; median 11 per evaluated user; 31 users have none and are not scored	53.7 % of ratings relevant; median 18 per evaluated user; 19 users have none and are not scored
History	≥ 1 earlier rating to be evaluated	3 users (168 ratings) not scored	3 users (547 ratings) not scored
Item filter	≥ 5 ratings in the fitting data	305 of 3,678 items dropped; 612 fitting ratings (0.07 %); 83 held-out ratings unreachable, 29 of them relevant → 3,373 kept	300 of 3,688 items dropped; 612 fitting ratings (0.06 %); 192 held-out ratings unreachable, 105 of them relevant → 3,388 kept

Filtering and preprocessing: We apply no activity filtering since there already exists a floor of 20 reviews because any stricter cutting would remove light users.[6] Thresholds are computed on the data the model is fitted on, so the parts are train to validate and train + validate to test. Users without earlier ratings give item-kNN nothing to start from and are exluded.[7] If the fitting data has items with less than 5 ratings they are dropped because they do not provide a significant enough signal.[8]

[3] Only batches 10 min within each other from a single user were considered.
[4] See page_time_by_batch_size.png
[5] See figures stars_by_batch_size.png
[6] Lecture 5.2 slide 6
[7] Lecture 1.2 slide 12
[8] Lecture 1.2 slides 10 and 12

Feedback treatment: The 5-star explicit rating system is used as implicit feedback for training.[9] All ratings count as positive interaction without considering the actual value of the rating. The previous part demonstrated that there is a high probability that users have done an active choice on whether to review a movie or not.[10] Because evaluation needs binary relevance, we turn the ratings of 4 or 5 to be considered as relevant in validation and testing.[11]

1.3 Train/validation/test split

We chose the split dates to be 1 Jan 2001 and 1 July 2001 as seen in Table 1. There is a high risk of a single month being non-representative for various reasons. Also the 80/10/10 split would have ended in the middle of the biggest spikes in new users (Figure 1). After the year 2000 only 6 new users rated for the first time which cuts less users out of the data.[12]

Table 2 - Split

Set	Period (UTC)	Ratings	Users	Users with no earlier ratings	Evaluated users
Train	25 Apr – 31 Dec 2000	904,757 (90.5 %)	6,034	–	–
Validation	1 Jan – 30 Jun 2001	47,314 (4.7 %)	926	3	892
Test	1 Jul 2001 – 28 Feb 2003	48,138 (4.8 %)	799	3	777

Sliding the split dates to 1 January 2001 and 1 July 2001 creates a split of 90.5/4.7/4.8. This was chosen because of a longer timespan for the validation period. Knowing that practically all new users will end up in the training data it was important to briefly analyse the post 2000 data for possible biases focused on having only recurring users. This can be possibly an advantage as it is highly improbable that the validating and testing populations would be separate populations (548 of the 799 test users are also validated). The test period of post 2000 means that only 19.5 % of users rate after 2000. They are more active (median ratings 151 vs. 77), so the results describe engaged returning users. Half of their ratings are still part of batches which means they are not rating only recently watched films and they rate blockbusters less frequently (top 1 % of movies receiving 5 % of reviews compared with 9 % in 2000).

On the downside, this temporal split means that the validation and test populations are considerably smaller than a conventional 80/10/10 split. We will probably have to introduce confidence intervals later. Also 90.3 % of training ratings are from first-week users which means that the validation and testing data is very different as it is mainly returning users.

1.4 Evaluation protocol

Only movies with at least five ratings in the fitting data could be recommended, and all models used the same set. Full ranking was chosen as the ranking setup, following the lecture materials (3.1). Each model ranked all candidate movies the user had not yet rated, instead of a random sample, since sampled evaluation can change the relative order of models. Computational complexity wasn't a major issue, given the size of the dataset.

The ratings were split by time into training, validation and test periods. All choices (k, similarity measure, Most Popular window and counting) were made on the validation period, and the test period was evaluated only once. Evaluated users had at least one rating in the fitting data and at least one relevant rating in the

[9] Lecture 1.1 slide 7
[10] Lecture 1.1 slide 6
[11] Lecture 3.1 slides 7 and 8
[12] Lecture 3.1 slide 4

evaluation period. Users without earlier ratings were excluded, since item-based kNN would not have anything to start from (Lecture 1.2). Movies the user had already rated were never recommended.

The used metrics were NDCG@10, Precision@10, Recall@10 and catalogue coverage, as suggested in the lecture materials (3.1). Differences in NDCG@10 between models were also reported with 95 % confidence intervals. More than one metric was used to have a broader view of the recommendations' quality. All metrics were computed for a list of ten recommendations per user, and a recommended item counted as relevant if the user rated it 4 or 5 stars in the evaluation period. In short, Precision@10 measures the share of the ten recommended items that were relevant, and Recall@10 the share of the user's relevant items that appeared in the list. NDCG@10 also accounts for order, giving more weight to relevant items near the top. Catalogue coverage measures the share of candidate items recommended to at least one user. The accuracy metrics were averaged over the evaluated users.

1.5 Baseline methods and settings

In item-based kNN, a user–item rating matrix was built from the fitting data and restricted to the candidate items. Each user's ratings were mean-centered (the user's average rating was subtracted from each rating), and unrated entries were set to zero. Adjusted cosine, i.e. cosine similarity computed on these mean-centered ratings, was used as the primary similarity measure. It was chosen following Sarwar et al. (2001), as mean-centering reduces the effect of users' different rating scales. The lecture materials (Lecture 1.2) also suggest cosine similarity for item–item comparisons. We compared it with plain cosine on raw ratings (results in results.csv). On the validation period, adjusted cosine outperformed plain cosine in Precision@10 and NDCG@10 at all three tested values of k, while plain cosine gave higher catalogue coverage.

Regarding the neighbourhood size, k = 5–200 (in steps of 5) was evaluated with the models fitted on the training period. NDCG@10 and Precision@10 improved as k increased from small values and reached a plateau at around k = 30 that continued up to k = 200. Recall@10 showed no clear trend, and catalogue coverage decreased as k grew. Based on this preliminary review, k = 20, 30 and 50 were selected for closer comparison (results.csv). Since accuracy no longer improved reliably beyond k = 30, we chose k = 30 among the plateau values, as it gave higher coverage than the larger values.

Most Popular was evaluated with windows of 7, 14, 30 and 60 days and the whole fitting data, each counting either all ratings or only ratings ≥ 4. On the validation period, counting only positive ratings improved NDCG@10 in every window (by 0.006–0.012). With positive ratings only, the 7-day and 14-day windows were statistically indistinguishable, and longer windows were clearly worse. We chose the 14-day window with positive ratings only, since it contains more ratings than the 7-day window, making the ranking more stable. 14-day window also has the best point estimate and a wider coverage than 7 days. However, this limits the amount of ratings considered so this might cause some issues later with overfitting too soon.

1.6 Results and analysis

Precision@10 0.172, Recall@10 0.076 and NDCG@10 0.193, about 0.05 higher in NDCG@10 than the selected Most Popular variant. The 14-day Most Popular variant, which performed best on validation, fell to an NDCG@10 of 0.145 on test, and the all-time ranking became the strongest popularity variant, so the recency effect seen on validation did not generalise. One possible reason is that few ratings were made just before the test cut, which makes a short window unstable. kNN also covered a far broader part of the catalogue (coverage 0.150 vs. 0.021), whereas popularity-based models recommend nearly the same items to everyone. Adjusted cosine consistently outperformed plain cosine on raw ratings (by about 0.02–0.03 NDCG@10). Full results can be seen in group's results -file.

2. Part 2 - Models and Honest Comparison

2.1 Models

Describe the additional models and paradigms. Clearly identify the self-implemented algorithm and explain its implementation.

2.2 Hyperparameter tuning and variability

Describe the validation-only tuning procedure, search spaces/budgets, random seeds, and confidence intervals or statistical testing.

2.3 Overall results

Present and discuss the main comparison. Explain important differences and avoid unsupported claims.

2.4 Disaggregated analysis

Report and interpret results broken down by at least two of the required dimensions: items with few vs. many training interactions, low- vs. high-activity users, popular vs. long-tail items, and early vs. late test period. Define any thresholds used and focus on findings that change or qualify the overall conclusions.

2.5 Sparsity ablation

Report and discuss the 100%, 50%, and 20% training-interaction experiments, including changes in the ranking between models.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

State Track A, B, or C and briefly motivate the selected intervention/extension.

3.2 Method

Describe only your selected track. The Track A/B/C guidance below is provided as instructions; delete the guidance for the two tracks you did not select before submission.

If Track A - Diversity and popularity bias

State the base recommender, selected re-ranking method, genre-vector item representation, trade-off parameter, and exact item-distance measure. Describe how the parameter was swept.

If Track B - Fairness

Define the release-decade provider proxy, report and justify the final decade boundaries and group sizes, state and justify the adopted fairness notion, describe the consumer groups based on MovieLens user attributes, and identify the stage and form of the mitigation.

If Track C - Group recommendations

State the selected synthetic group type and justify the chosen group size (3-5). Describe the three aggregation strategies. Identify which strategy was extended to the sequential scenario and explain how satisfaction is carried forward between recommendation rounds.

3.3 Results and trade-offs

Present the required track-specific results. Track A: report intra-list diversity, catalogue coverage, Gini and calibration error across the parameter sweep, and discuss the accuracy-diversity frontier. Track B: report position-weighted exposure against catalogue share, per-group consumer-side accuracy, and the fairness-performance frontier. Track C: compare group satisfaction, disagreement and minimum member satisfaction across the three aggregation strategies, then analyse how member and group satisfaction develop in the sequential scenario.

3.4 Cost and interpretation

State what is gained, what is lost, and who bears the cost where applicable. Explain what operating choice or strategy you would select and why.

3.5 "One number we cannot explain"

Identify one result you do not fully understand and describe how you would investigate it.

4. Discussion and Conclusions

Summarize the main findings across the project. Discuss limitations, failed approaches where relevant, and what the experiments support.

5. Contribution Statement

For each team member, provide two or three sentences describing their contribution.

6. Use of AI Assistants

152286617: Claude Opus 5.5 wrote all code required for this course as instructed and coached by the student. While it would have been possible to complete the course without AI assistance in coding it was more interesting to see how deep into the data it was possible to dive when more time was left over to understanding the results. The goal was to get deeper into the data rather than focus on writing code. All written text and analysis on this document were written by the student completely.

153823257: Claude Sonnet 5.5 was used for sparring and debugging code.

References

List references used in the report. References do not count toward the page limit.

Sarwar, Badrul, et al. “Item-Based Collaborative Filtering Recommendation Algorithms.” *Proceedings of the 10th International Conference on World Wide Web* [New York, NY, USA], 2001, pp. 285–95, https://doi.org/10.1145/371920.372071.